Informe Técnico: Recomendaciones Agronómicas de Croply

1. Resumen general y Propósito del Módulo

El módulo de Inteligencia Artificial de **Croply** fue diseñado para resolver el problema central relevado en pequeños y medianos productores de los oasis irrigados de Mendoza (como el caso estudiado en Primavera, Guaymallén): la toma de decisiones críticas (cuándo regar, cuándo aplicar agroquímicos o cómo proteger el cultivo ante contingencias climáticas) basada únicamente en la intuición o en pronósticos regionales genéricos.

Para cubrir las historias de usuario **HU-NA-01** (*Recibir alertas climáticas automáticas*) y **HU-NA-03** (*Visualizar recomendaciones agrícolas generadas por IA*), se construyó una **Arquitectura Híbrida de Dos Etapas** en Python 3.13 con FastAPI:

1. **Motor Analítico y Predictivo (`analyzer.py`):** Combina reglas deterministas de alertas meteorológicas con un modelo supervisado **LightGBM** y explicabilidad **SHAP (`TreeExplainer`)** que clasifica en milisegundos (~ 25 ms) la acción agronómica exacta a tomar sobre la parcela.
2. **Orquestador de Lenguaje Natural (`recommender.py`):** Consume en memoria el diagnóstico estructurado del analizador y utiliza **Gemma 4 sobre Ollama** (con validación estricta Pydantic y un *fallback* determinista de contingencia) para redactar los textos empáticos y accionables para el productor sin riesgo de alucinaciones.

2. Decisiones de Diseño: ¿Por qué lo hicimos así y qué ventajas trae?

2.1. ¿Por qué descartamos los datasets públicos tradicionales ("Crop Recommendation")?

Durante la Fase 1 de investigación en repositorios internacionales (Kaggle, UCI, Zenodo, Mendeley Data), detectamos que la mayoría de los datasets públicos de machine learning aplicado a la agricultura responden a la pregunta **"¿Qué**

cultivo conviene plantar en este suelo virgen?" a partir de Nitrógeno (N), Fósforo (P), Potasio (K), pH y lluvia anual.

- **Por qué no servían para Croply:** El productor apuntado en Croply **ya tiene un cultivo implantado en la parcela**. Nuestro problema no es de selección de cultivo pre-siembra, sino de **monitoreo continuo y manejo operativo diario** (programación de riego, ventanas de pulverización y alertas fenológicas).

2.2. ¿Por qué construimos un Dataset?

Ante la inexistencia mundial de un dataset público que combine las hortalizas típicas de Cuyo con telemetría IoT y etiquetas de acción operativa, diseñamos un generador científico híbrido (`build_hybrid_dataset.py`) sustentado en tres pilares reales:

1. **Clima Real de Mendoza (Open-Meteo Historical API):** Se descargaron **17.544 horas reales** (2 años completos: 2024–2025) de Guaymallén, Mendoza, a través de ERA5-Land (conjunto de datos de reanálisis climático).
    - *Ventaja clave:* El modelo entrenó con las mismas distribuciones estadísticas, unidades y fenómenos extremos reales de Cuyo (heladas de hasta $-7.7°\mathrm{C}$, olas de calor de $42.3°\mathrm{C}$ y ráfagas de Viento Zonda de $54.5\ \mathrm{km/h}$) que el backend en producción recibe desde la API de pronóstico de Open-Meteo.
2. **Fenología y Calendario del INTA + Manual FAO-56 (estándar internacional publicado por la Organización de las Naciones Unidas para la Alimentación y la Agricultura):** Se parametrizaron los **6 cultivos más representativos de Cuyo** (**Tomate, Ajo, Lechuga, Cebolla, Pimiento y Papa**) respetando sus meses de siembra y duración de ciclo del *Calendario de Siembra del INTA*, junto con sus coeficientes de cultivo (K_c) y fracciones de agotamiento permisible de agua (MAD / p) del estándar internacional FAO-56.
3. **Características de Suelo Mendocino y Reglas de Campo:**
    - Se calibró el suelo como **Franco** ($\theta_{CC} = 30.0\%$ de Capacidad de Campo, $\theta_{PMP} = 14.0\%$ de Punto de Marchitez Permanente y $AU = 16.0\%$ de Agua Útil).

- Se modeló la regla empírica relevada en las entrevistas de finca: **una lluvia leve (< 5 mm) humedece el follaje y el aire, pero no infiltra hasta la zona radicular profunda**.

2.3. ¿Por qué separar `analyzer.py` (LightGBM) de `recommender.py` (Gemma 4)?

Si le enviáramos los datos crudos de los sensores directamente a un LLM (Gemma 4) para que decida qué hacer, enfrentaríamos tres problemas graves:

1. **Alucinaciones agronómicas:** Un LLM puede confundir umbrales numéricos de humedad o recomendar pulverizar con viento Zonda.
2. **Latencia excesiva:** Un LLM local tarda entre 3 y 15 segundos por inferencia. Si una parcela está en estado óptimo, llamarlo desperdicia CPU/GPU.
3. **Falta de determinismo ante caídas (`[ERR-01]`):** Si Ollama se apaga o supera el *timeout*, el productor se quedaría sin recomendación.

Ventajas de nuestra arquitectura desacoplada:

- **Velocidad:** LightGBM evalúa la parcela y calcula los valores SHAP en ~ 25 ms (consumiendo apenas ~ 25 MB de RAM).
- **Ahorro de cómputo (Regla de Estado Vacío de `HU-NA-03`):** Si `analyzer.py` dictamina `ESTADO_OPTIMO_SIN_ACCION` y no hay alertas climáticas activas, `requiere_intervencion_llm` es `False`. En ese caso, `recommender.py` **no invoca a Ollama** y devuelve instantáneamente el mensaje requerido por la interfaz: *"No hay recomendaciones activas en este momento."*.
- **Alta disponibilidad (`Fallback Determinista`):** Si Ollama demora más de 30 segundos o no está disponible (`[ERR-01]`), el servicio conmuta automáticamente al texto técnico precalculado por `analyzer.py` (`fallback_determinista_inta`), logrando **100% de disponibilidad operativa**.

3. Explicación de los Tipos de Clasificaciones y Alertas del Sistema

El servicio realiza **dos evaluaciones complementarias** en cada ejecución:

3.1. Predicción A: Alertas Climáticas Automáticas (`HU-NA-01`) y Riesgo Fenológico

Definimos **Riesgo Fenológico** como la probabilidad de que un cultivo sufra daños o pérdidas por coincidir una de sus etapas de desarrollo más sensibles (como la brotación, la floración o el cuajado del fruto) con un evento meteorológico adverso, como una helada tardía, una granizada o una ola de calor.

Evalúa el pronóstico meteorológico de las próximas 24–48 hs junto con la sensibilidad de la etapa actual del cultivo (`inicial`, `desarrollo`, `media`, `final`):

1. `ALERTA_HELADA` **(Severidad Alta):** Temperatura mínima pronosticada $\leq 0.0°\text{C}$.
2. `ALERTA_HELADA_AGRONOMICA` **(Severidad Alta):** Temperatura mínima por encima de $0°\text{C}$ pero dentro del umbral de daño celular ($\leq T_{\text{helada_cultivo}} + 1.5°\text{C}$) durante etapas críticas como trasplante/implantación (`inicial`) o floración/cuaje (`media`).
3. `ALERTA_LLUVIA_INTENSA` **(Severidad Media):** Precipitación acumulada pronosticada $\geq 30.0\ \text{mm}$ en 24 hs.
4. `ALERTA_VIENTO_FUERTE_ZONDA` **(Severidad Alta):** Velocidad de viento $\geq 50\ \text{km/h}$, ráfagas $\geq 55\ \text{km/h}$, o combinación típica de **Viento Zonda en Cuyo** (viento $\geq 35\ \text{km/h}$ con humedad relativa $< 20\%$ y temperatura $> 28°\text{C}$).
5. `ALERTA_OLA_CALOR` **(Severidad Alta):** Temperatura máxima pronosticada $\geq 35.0°\text{C}$.

3.2. Predicción B: Clasificación Multi-Clase de Acción Agronómica (`HU-NA-03`)

El modelo **LightGBM** clasifica el estado de la parcela en una de **7 acciones mutuamente excluyentes**, acompañadas de su **Severidad (`Alta`, `Media`, `Baja`)**:

Clase (accion_recomendada)	Severidad	Qué significa agronómicamente	Cuándo se dispara
`REGAR_URGENTE`	`Alta`	Estrés hídrico severo en zona radicular con riesgo de marchitez o aborto floral.	La humedad del suelo cae cerca del Punto de Marchitez ($\leq 18\%$) o hay déficit hídrico en plena floración con calor ($> 32°\text{C}$).
`REGAR_PROGRAMADO`	`Media`	Agotamiento del agua fácilmente	La humedad del suelo desciende

		utilizable; requiere reposición preventiva.	por debajo del umbral crítico de la etapa fenológica ($\theta_{\text{crit}} \approx 24\%–26.8\%$), sin llegar aún a emergencia.
`SUSPENDER_RIEGO`	`Media` / `Alta`	Riesgo de asfixia radicular (anoxia), lavado de nutrientes o pudrición de bulbo.	Tres causas posibles discriminadas en el código: (1) Suelo saturado ($> 31.5\%$), (2) Lluvia abundante inminente ($\geq 20–25\ \text{mm}$), o (3) Etapa final de secado pre-cosecha en **Ajo y Cebolla**.
`VENTANA_OPTIMA_CURAR`	`Baja`	Condiciones meteorológicas ideales para pulverizar agroquímicos con máxima eficacia.	Toca curar (≥ 14 días desde la última aplicación o alta presión fúngica) **Y** hay viento suave ($3–12\ \text{km/h}$, sin inversión térmica ni deriva), temperatura moderada ($14–25°\text{C}$), ΔT psicrométrico entre 2 y $8°\text{C}$ y sin lluvias próximas.

`POSTERGAR_CURACION_CLIMA`	`Media` / `Alta`	Prohibición operativa de aplicar agroquímicos por riesgo ambiental o pérdida de producto.	Se requiere curar, pero hay **viento** $> 15\ \mathrm{km/h}$ (deriva química), $\Delta T > 10^{\circ}\mathrm{C}$ (evaporación violenta de la gota antes de tocar la hoja), calor $> 28^{\circ}\mathrm{C}$ o lluvia próxima ($> 1.5\ \mathrm{mm}$).
`CORREGIR_PH_SUELO`	`Baja` / `Media`	Bloqueo nutricional por acidez o alcalinidad edáfica fuera de rango.	El sensor de `ph_suelo` está activo y reporta valores fuera del rango óptimo del cultivo (ej. > 6.5 en Papa con riesgo de sarna, o fuera de $[6.0, 6.8]$ en Tomate).
`ESTADO_OPTIMO_SIN_ACCION`	`Baja`	El cultivo vegeta dentro de los parámetros ideales para su etapa fenológica.	Humedad en confort, pH normal y sin tratamientos pendientes.

4. Tolerancia a Sensores Faltantes y Guardarraíles de Producción

En la realidad de las fincas de Cuyo (y según las historias **HU-FP-03** y **HU-IoT-02**), no todas las parcelas tienen los 5 sensores comprados, o una sonda puede quedar temporalmente *"Sin transmisión"* (`null`). Implementamos **4 mecanismos de blindaje**:

1. **Entrenamiento con *Missing Sensors* (`np.nan`):** El dataset se generó ocultando deliberadamente el **20%** de los sensores de pH, el **15%** de radiación solar y el **8%** de humedad de suelo, repartiendo equitativamente los escenarios restantes para no sesgar al árbol.

2. **Respaldo Ambiental Automático:** Si el sensor ambiental (`temp_ambiente_c` o `humedad_ambiente_pct`) envía `null`, `analyzer.py` no rechaza la petición (evitando errores HTTP 422) y utiliza la temperatura media estimada del pronóstico de la finca y una humedad base del 50% para poder evaluar el suelo.
3. **Máscara de Viabilidad Física (`apply_feasibility_mask`):** Antes de emitir el veredicto, el código anula a `0.0` la probabilidad de cualquier acción cuyo sensor físico sea `null` (ej. prohibido predecir `CORREGIR_PH_SUELO` sin sensor de pH, y prohibido predecir `REGAR_*` sin sensor de humedad de suelo) y re-normaliza las probabilidades restantes a `1.0`.
4. **Activación Adaptativa ante Alertas en Estado Óptimo:** Si el suelo está en `ESTADO_OPTIMO_SIN_ACCION` pero el pronóstico dispara una alerta grave (ej. `ALERTA_HELADA` con $-2^{\circ}\mathrm{C}$), el sistema activa la recomendación preventiva para que el productor proteja el lote.

5. Cómo Leer e Interpretar los Resultados

5.1. Cómo leer las métricas de entrenamiento del Modelo (`v1.3.0`)

En la evaluación sobre **3.600 muestras de prueba no vistas (Test Set)**:

- **Macro F1-Score (`0.9511`):** Es el promedio armónico entre *Precision* (qué porcentaje de las veces que el modelo recomienda una acción tiene razón) y *Recall* (qué porcentaje de los casos reales de esa acción logra detectar), dándole el mismo peso a las 7 clases. Supera holgadamente el estándar de producción (≥ 0.85) sin caer en un 1.00 artificial gracias al ruido gaussiano realista ($\sigma = 0.80\%$ en suelo) inyectado después del etiquetado físico.
- **Detalle por clase:** La menor puntuación ocurre entre `REGAR_PROGRAMADO` (`F1 = 0.86`) y `REGAR_URGENTE` (`F1 = 0.92`), lo cual es física y agronómicamente lógico porque ambas clases comparten una frontera continua de humedad alrededor del 18% donde el ruido del sensor capacitivo genera un leve solapamiento gradual.

5.2. Cómo leer el JSON que devuelve el Microservicio (`/api/v1/ai/evaluate-parcel`)

Cuando el backend consulta al servicio de IA, recibe un JSON con esta estructura clave:

- `accion_recomendada` y `severidad`: La decisión principal (ej. `"REGAR_URGENTE"`, `"Alta"`). La severidad le sirve a la pantalla *"Mi finca"* (`PAN-AF-01`) para ordenar y mostrar en la tarjeta resumen la recomendación más urgente entre todas las parcelas.
- `sensor_origen_principal`: Indica qué sensor físico o variable meteorológica disparó la decisión (ej. `"humedad_suelo_pct"`, `"viento_kmh"` o `"pronostico_lluvia_24h"`). Se calcula en vivo mediante **SHAP**, descartando sensores en `null` y variables estáticas.
- `explicabilidad_top3_factores`: Lista los 3 factores físicos principales con su `"valor"` real y su `"shap_attribution"`.
  - *Cómo leer* `shap_attribution`*:* Un valor positivo alto (ej. `+9.55` o `+4.19`) significa que esa lectura empujó fuertemente al modelo a elegir esa acción. Además, el motor agrupa variables semánticamente equivalentes (no repite `humedad_suelo_pct` y `deficit_hidrico`) y excluye metadatos internos (`cultivo_enc`, `hora_del_dia`).
- `recomendacion_vigente`: Objeto validado con los 3 textos que pide la base de datos en **HU-NA-03**:
  - `titulo_corto` (≤ 40 caracteres): Para el encabezado de la tarjeta en `PAN-AF-01` y `PAN-AF-03`.
  - `texto_breve` (≤ 120 caracteres): Subtítulo conciso con el valor del sensor principal.
  - `texto_explicativo_extendido`: Párrafo de 2 a 3 oraciones con el fundamento agronómico y la acción sugerida (visible en el detalle de parcela `PAN-AF-03`).
- `fuente_generacion`: Permite auditar si el texto fue redactado por `"ollama_gemma4"`, por `"fallback_determinista_inta"` (cuando Ollama no está disponible) o por `"estado_optimo_sin_accion"`.

6. Guía de Uso de las Herramientas y Comandos de Ejecución

Todo el flujo de trabajo está contenido en la carpeta `src_ml/` y es 100% reproducible desde la terminal:

Paso 1: Descargar el Clima Histórico Real de Mendoza (Opcional, ya descargado)

Consulta la API histórica de Open-Meteo para Guaymallén y genera `raw_datasets/clima_mendoza_openmeteo.csv`:

Bash

```
python src_ml/fetch_mendoza_weather.py
```

Paso 2: Generar el Dataset Híbrido Calibrado (`18.000` muestras)

Cruza el clima real con las fichas del INTA, simula el balance hídrico de suelo Franco, inyecta porcentajes de sensores desconectados (`NaN`) y guarda `raw_datasets/croply_training_dataset_cuyo.csv`:

Bash

```
python src_ml/build_hybrid_dataset.py
```

Paso 3: Entrenar, Evaluar y Serializar el Modelo LightGBM

Entrena el clasificador, aplica la máscara de viabilidad física en test, corre las pruebas unitarias de sensores faltantes y exporta el artefacto `src_ml/models/croply_classifier_v1.joblib`:

Bash

```
python src_ml/train_baseline.py
```

Paso 4: Ejecutar la Suite de Pruebas de Integración End-to-End (5 Flujos)

Valida el endpoint de salud (`/api/v1/ai/health`) y los 4 escenarios críticos de negocio (Estrés Hídrico, Sensores en `null` + Viento Fuerte, Estado Vacío Óptimo y Alerta de Helada en Estado Óptimo):

Bash

```
python src_ml/test_service_e2e.py
```

Paso 5: Levantar el Microservicio FastAPI en Producción / Desarrollo Local

Inicia el servidor web en el puerto `8000` (con documentación interactiva Swagger automática en `http://localhost:8000/docs`):

Bash

```
uvicorn src_ml.main:app --host 0.0.0.0 --port 8000 --reload
```

(Opcional: Para usar la redacción en vivo con Gemma 4 en lugar del fallback determinista, tener corriendo el servidor local de Ollama en `http://localhost:11434` con la variable de entorno `OLLAMA_MODEL=gemma4`).